Using Reusable Development Skills

A practical guide to context-aware AI workflows

Six validated skills for Java 21 and Spring Boot development, testing, review, diagnosis, refactoring and security.

Companion guide to the AI Engineering Starter Kit
Version 1.0 | 2026

Contents

Skills turn standards into repeatable work

A skill is a version-controlled procedure that tells an AI assistant how to perform a task consistently. Project facts do not live in the skill. They live in the repository's Markdown context.

Layer	Responsibility	Example
Project context	What is true for this repository	.ai/architecture.md
Reusable skill	How to perform a type of work	create-spring-feature/SKILL.md
Developer request	The specific outcome required now	Add customer search by email
Automated gates	Objective enforcement and evidence	mvn verify and CI
Human review	Accountability and risk acceptance	Pull-request approval

Why this separation matters

- One skill can work across many repositories without copying project rules.
- Architecture, security and testing standards remain reviewable beside the code.
- Updating a Markdown rule changes future skill behaviour without editing every skill.
- Tool-specific wrappers can point to one authoritative context set, reducing drift.

Install and invoke the skills

The starter repository stores all six definitions under `skills/`. Copy each complete folder into the active Codex skills directory or distribute it through your approved internal mechanism.

Repository location

```
C:\Tools\Workspace\Auth\ai_coding\skills
```

Skill folder contract

```
skills/
create-spring-feature/
SKILL.md
agents/openai.yaml
```

Explicit invocation

```
Use $create-spring-feature to add customer search by email.
```

Invocation checklist

- Run the request from the intended repository so the skill can locate AGENTS.md and .ai/project.md.
- Name the outcome and acceptance criteria; do not repeat standards already held in .ai files.
- State important scope limits, such as no public API break or no new dependency.
- Ask for evidence: tests run, quality gates, changed files, assumptions and residual risk.

$create-spring-feature

Build a complete, tested Spring Boot vertical slice while respecting repository architecture and policy.

Use it for

- New REST endpoints or use cases
- DTO, service, repository or migration changes
- Validation, OpenAPI and error handling

Context loaded before work begins

`AGENTS.md`, `.ai/project.md`, `.ai/architecture.md`, `.ai/coding-standards.md`, `.ai/api-guidelines.md`, `.ai/security.md`, `.ai/testing.md`, `.ai/business-rules.md`, `.ai/domain-model.md`, `.ai/dependencies.md`

Example invocation

```
Use $create-spring-feature to add:
GET /api/v1/customers/search?email={email}

Return 404 when absent, validate the email, update OpenAPI,
add unit and integration tests, and do not add dependencies.
```

Expected behaviour

- Restate acceptance criteria and risk before editing.
- Implement the smallest coherent vertical slice.
- Add tests with real assertions and run relevant Maven verification.
- Report changed files, evidence and residual risk.

$generate-java-tests

Create behavioural tests that prove outcomes rather than merely exercising code.

Use it for

- JUnit 5 and AssertJ unit tests
- Mockito collaborator isolation
- Spring MVC, repository or Testcontainers integration tests
- Regression tests for failures

Context loaded before work begins

`AGENTS.md`, `.ai/project.md`, `.ai/testing.md`, `.ai/architecture.md`, `.ai/coding-standards.md`, `.ai/security.md`, `.ai/business-rules.md`

Example invocation

```
Use $generate-java-tests to inspect CustomerService and add
missing positive, negative and boundary tests.

Do not change production behaviour without reporting the defect first.
```

Expected behaviour

- Identify observable behaviours before choosing tests.
- Mock external boundaries only.
- Place at least one meaningful value, state, result or exception assertion in every test.
- Run targeted tests and report coverage gaps.

$review-java-change

Perform an independent, evidence-based review against the repository's own standards.

Use it for

- Uncommitted changes
- Pull requests or patches
- AI-generated Java and Spring code
- Test-quality and compatibility reviews

Context loaded before work begins

`AGENTS.md`, `.ai/architecture.md`, `.ai/coding-standards.md`, `.ai/api-guidelines.md`, `.ai/security.md`, `.ai/testing.md`, `.ai/governance.md`, `.ai/business-rules.md`

Example invocation

```
Use $review-java-change to review the current changes.
Prioritise correctness, authorization, API compatibility and test quality.
Do not edit files. Cite exact file and line references.
```

Expected behaviour

- Lead with findings ordered by severity.
- Explain the credible failure scenario and impact.
- Identify tests without real outcome assertions.
- Separate blockers from optional improvements.

$diagnose-java-failure

Find the root cause of a build, test, runtime, database or CI failure before changing code.

Use it for

- Maven or compiler failures
- Spring startup and configuration problems
- Failed or flaky tests
- Database, container and CI errors

Context loaded before work begins

`AGENTS.md`, `.ai/project.md`, `.ai/architecture.md`, `.ai/testing.md`, `.ai/security.md`, `.ai/dependencies.md`

Example invocation

```
Use $diagnose-java-failure to investigate:
CustomerServiceTest expected a customer but received CustomerNotFoundException.

Reproduce and identify the root cause. Do not implement a fix yet.
```

Expected behaviour

- Record the exact symptom and narrowest reproduction.
- Distinguish primary failure from cascading messages.
- Test ranked hypotheses and show evidence.
- Recommend the smallest fix plus a regression test.

$refactor-java-code

Improve maintainability while explicitly preserving behaviour and public contracts.

Use it for

- Remove duplication
- Improve names and method boundaries
- Separate responsibilities
- Modernise Java without adding features

Context loaded before work begins

`AGENTS.md`, `.ai/project.md`, `.ai/architecture.md`, `.ai/coding-standards.md`, `.ai/testing.md`, `.ai/api-guidelines.md`, `.ai/business-rules.md`, `.ai/domain-model.md`

Example invocation

```
Use $refactor-java-code to remove duplicated customer-to-DTO mapping.

Preserve HTTP and database behaviour, add no dependencies,
and keep the change limited to mapping.
```

Expected behaviour

- Freeze protected behaviour and contracts.
- Add characterization tests when protection is weak.
- Keep refactoring separate from feature work.
- Report behaviour-preservation evidence and unverified contracts.

$secure-java-review

Assess credible security and abuse risks across code, configuration, data and dependencies.

Use it for

- Authentication and authorization review
- Input, database and SSRF risk
- Secrets, logging and error leakage
- Dependency and supply-chain review

Context loaded before work begins

`AGENTS.md`, `.ai/security.md`, `.ai/governance.md`, `.ai/project.md`, `.ai/architecture.md`, `.ai/api-guidelines.md`, `.ai/dependencies.md`, `.ai/business-rules.md`

Example invocation

```
Use $secure-java-review to assess the customer search endpoint.

Review authorization, enumeration, logging, query safety and abuse risk.
Do not modify code. Return severity-ranked findings.
```

Expected behaviour

- Map assets, actors, entry points and trust boundaries.
- Demonstrate a credible path from input to impact.
- Include preconditions, evidence and remediation.
- Escalate high-risk ambiguity to the correct owner.

Worked example: customer search feature

Use separate skill invocations as explicit quality stages. This preserves independence between creation, testing and review.

Stage	Invocation	Human decision
1. Build	$create-spring-feature	Confirm contract and scope
2. Test	$generate-java-tests	Confirm behaviours and edge cases
3. Review	$review-java-change	Resolve blockers
4. Secure	$secure-java-review	Accept or remediate risk
5. Verify	mvn verify and Git hooks	Approve evidence
6. Merge	Pull-request approval	Accountable release decision

Build request

```
Use $create-spring-feature to add GET /api/v1/customers/search?email={email}.
Validate email, return 404 when absent, update OpenAPI, include tests and add no dependency.
```

Independent review request

```
Use $review-java-change to review the customer search change.
Do not edit files. Focus on data exposure, API compatibility and tests with real assertions.
```

Write effective skill requests

The skill defines the method. Your request defines the outcome.

Include	Good example	Why
Outcome	Search customers by email	Defines success
Acceptance criteria	404 when absent	Makes behaviour testable
Scope	Customer package only	Controls unintended edits
Constraints	No new dependencies	Preserves architecture and supply chain
Evidence	Run targeted tests and mvn verify	Makes completion auditable
Authority	Diagnose only; do not fix	Separates analysis from mutation

Avoid

- Vague requests such as 'make the code better' without a protected behaviour or scope.
- Repeating large standards blocks that may conflict with version-controlled .ai context.
- Combining feature creation, broad refactoring and dependency upgrades in one change.
- Asking the same AI run to create, approve and merge its own output.

Review the skill's output

A completed invocation should make its evidence and limitations visible.

- **Context:** applicable repository guidance was located and read.
- **Intent:** requirements, assumptions, scope and risk are explicit.
- **Change:** changed files and design decisions are concise and traceable.
- **Tests:** every test has a meaningful outcome assertion.
- **Verification:** exact commands and results are reported.
- **Limitations:** unverified behaviour, missing access and residual risk are not hidden.
- **Review:** the appropriate human or security owner remains responsible for approval.

Stop conditions

- Conflicting Markdown rules or an unavailable required context file.
- A missing requirement that changes public API, authorization, data model or architecture.
- Potential exposure of secrets, personal data, client data or proprietary material.
- A high-risk security ambiguity without an accountable decision owner.
- A requested action outside the developer's or tool's authority.

Troubleshooting

Problem	Likely cause	Resolution
Skill is not recognised	Folder is not installed in the active skills directory	Install the complete folder and restart or refresh the Codex session
Context is not found	Request started outside the repository	Open the repository root containing AGENTS.md or .ai/project.md
Output conflicts with standards	Context files disagree or contain copied rules	Make .ai authoritative, remove duplication and resolve the conflict
Tests contain verify only	Testing rule was not enforced	Invoke $generate-java-tests and require a meaningful outcome as
Review edits the code	Authority was ambiguous	State 'review only; do not edit files'
Verification cannot run	Missing Java, Maven, Docker or service	Report the gap; do not claim a pass; run CI or provide the depend

Governance reminder

Reusable skills improve consistency; they do not transfer accountability to the model. Apply the repository's risk classification, data-handling policy, approval gates and audit requirements to every AI-assisted change.

Quick-reference prompts

Create

```
Use $create-spring-feature to implement [outcome]. Acceptance criteria: [behaviours]. Scope: [files/boundary]. Constraints: [compatibility/dependencies]. Run [evidence].
```

Test

```
Use $generate-java-tests to cover [component]. Include [positive/negative/boundary] behaviours. Do not change production behaviour without reporting the defect.
```

Review

```
Use $review-java-change to review [diff/PR]. Do not edit files. Prioritise [risks] and cite exact file and line references.
```

Diagnose

```
Use $diagnose-java-failure to reproduce and identify the root cause of [failure]. Diagnose only; do not implement a fix.
```

Refactor

```
Use $refactor-java-code to improve [problem]. Preserve [contracts/behaviour], add no dependencies and limit the change to [scope].
```

Secure

```
Use $secure-java-review to assess [change]. Review [trust boundaries/risks]. Do not edit code; return severity-ranked findings.
```

The skill supplies the repeatable method. The repository supplies the standards. The developer supplies the requirement and remains accountable for the result.